Transform: Cleaning, Conforming & Curating Data for Reliable Analytics

Silver layer, dbt and Containerization

Agenda

01 SCD

02 What is dbt?

03 Core concepts

04 Project structure

05 Writing models

06 Testing and Docs

07 Running dbt

08 Docker and Containers

Maybe silver is just the metal?

The Silver Layer is the second tier of the Medallion Architecture. It receives raw Bronze data and transforms it into clean, validated, and conformed records ready for downstream consumption.

Key Responsibilities

Data Cleansing

Remove nulls, fix typos, standardize formats, and deduplicate records.

Validation & Quality

Apply schema checks, business rules, and referential integrity constraints.

Transformation

Normalize, join, enrich, and map data to a unified semantic model.

Partitioning & Storage

Partition by date or entity for efficient querying and incremental loads.

Common Technology and Patterns

Processing

dbt · Databricks · Apache Spark

Storage Format

Parquet · Delta Lake · Apache Iceberg

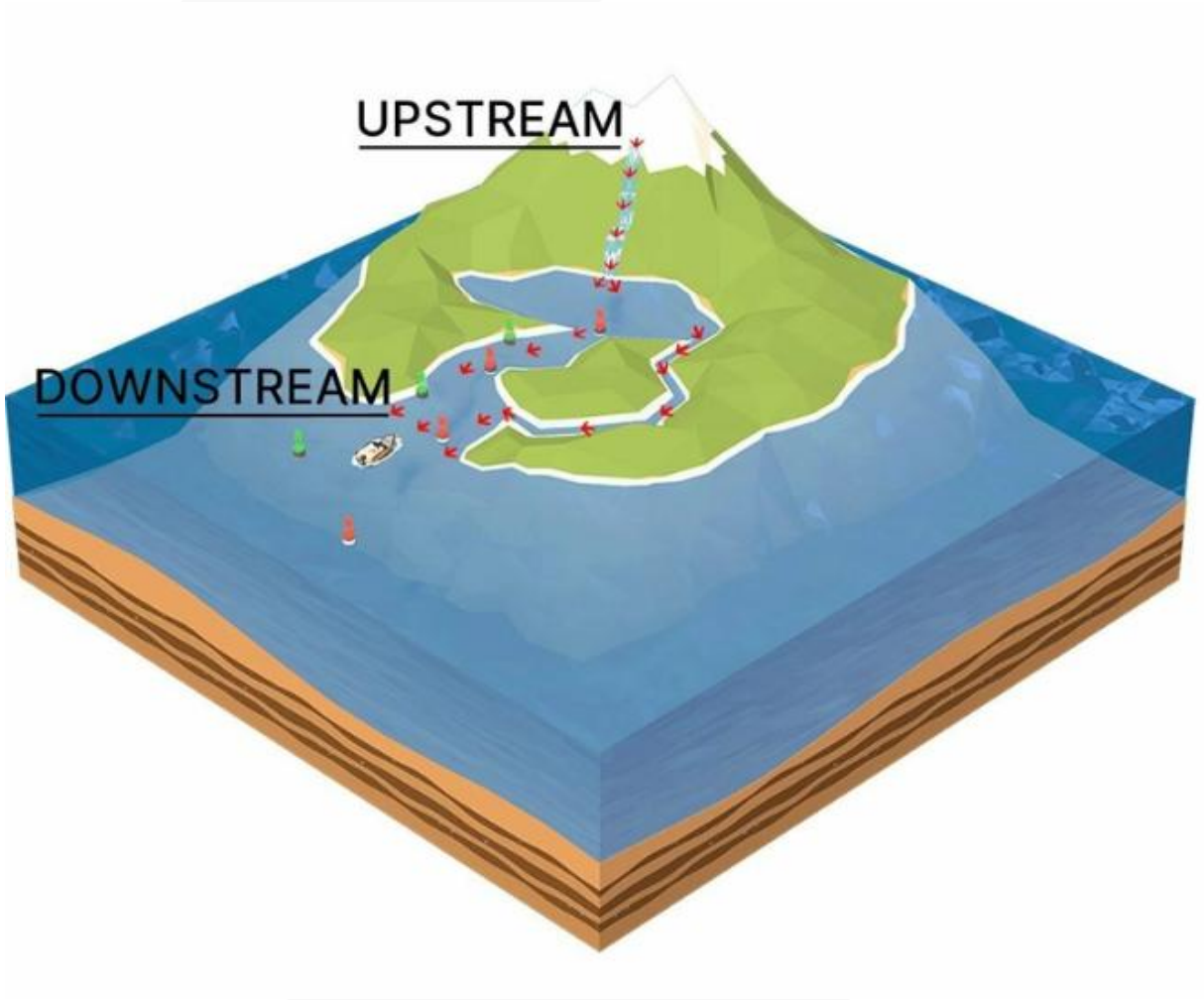
Orchestration

Apache Airflow · Prefect · Azure Data Factory

Quality

Great Expectations · Soda · dbt tests

Terminology for addressing to Data Pipelines



Best practices and key takeaways

01 Idempotent pipelines: re-running Silver jobs should yield the same result.

02 Track data lineage: know exactly which Bronze source each Silver record came from.

03 Fail fast: reject bad records early with clear error logs, don't silently drop data.

04 Incremental over full loads: process only new or changed records for scalability.

05 Document contracts: define schemas and SLAs between Silver and Gold consumers.

levi nine

How data warehouses keep track of information that changes over time

SCD - Slowly Changing Dimensions

What is a dimension

In a data warehouse, a **dimension table** stores descriptive attributes about business entities, customers, products, employees, stores.

The problem: these attributes change. A customer moves house. A product gets a new category. An employee changes department.

A Slowly Changing Dimension (SCD) is a technique for managing and tracking those changes over time.

Example: Customer Dimension

Cust ID	Name	City
101	A. Chen	Boston
102	M. Diaz	Austin

A. Chen moves to Denver. Do we overwrite Boston, or keep a record of it?
That decision is what SCDs are all about.

The Simplest Approaches

Type 0 and Type 1 don't preserve history, they differ only in whether change is allowed.

Type 0 - Retain Original

The attribute is fixed once loaded. No updates are ever applied, even if the real-world value changes.

Example: a customer's original signup date never changes.

Type 1 - Overwrite

The old value is simply replaced with the new one. Simple and cheap, but all history is lost.

Example: correcting a misspelled name, the old spelling is gone for good.

Type 2 - Add a New Row

The most common SCD in practice, it keeps full history by inserting a new row for every change.

How it works

- Each version of a record gets its own row
- Start Date / End Date mark when each version was valid
- A Current Flag shows which row is active today
- A Surrogate Key uniquely identifies each version

Example: A. Chen's address history

Key	Cust ID	City	Start Date	End Date	Current
1001	101	Boston	2019-01-05	2023-06-30	No
1002	101	Denver	2023-07-01	NULL	**Yes**

Both rows stay in the table forever — nothing is deleted, so we can always answer "where did this customer live in March 2021?"

Surrogate Key

In a Slowly Changing Dimension (SCD), a **surrogate key** is a unique, system-generated ID number given to each row in a data warehouse table. It has no real-world meaning. It replaces the source system's natural business ID (like a standard Customer ID or Product Code) as the table's primary key.

Original Database Record:

Surrogate Key (Primary Key)	Business Key	Name	City	Start Date	End Date
101	55	John Doe	New York	2023-01-01	NULL

Surrogate Key (Primary Key)	Business Key	Name	City	Start Date	End Date
101	55	John Doe	New York	2023-01-01	2025-06-30
102	55	John Doe	Seattle	2025-07-01	NULL

Type 3 — Add a New Column

A middle ground: keep just the previous value alongside the current one, in the same row.

Instead of adding rows, Type 3 adds **extra columns,** one for the current value, one for the previous value.

It only remembers one step back: useful for “before vs. after” comparisons, but not a full history.

Example: Sales Region Assignment

Rep ID	Current Region	Previous Region
205	West	Midwest

If the rep moves again, “Previous Region” is overwritten, only the most recent change is remembered.

Comparing the Types

Type	Keeps History?	Storage Cost	Common Use Case
Type 0 - Retain Original	No (locked)	Lowest	Values that must never change (e.g. original signup date)
Type 1 - Overwrite	No	Lowest	Fixing errors, minor attributes no one needs to track
Type 2 - Add New Row	Yes, full history	Highest	Attributes that matter for historical reporting (address, department)
Type 3 - Add New Column	Only 1 prior value	Low–Medium	Simple before/after comparisons

Key takeaways

Dimensions describe business entities, and their attributes change over time.

SCDs are simply strategies for handling those changes in a data warehouse.

Type 0 locks values; Type 1 overwrites them, both lose history.

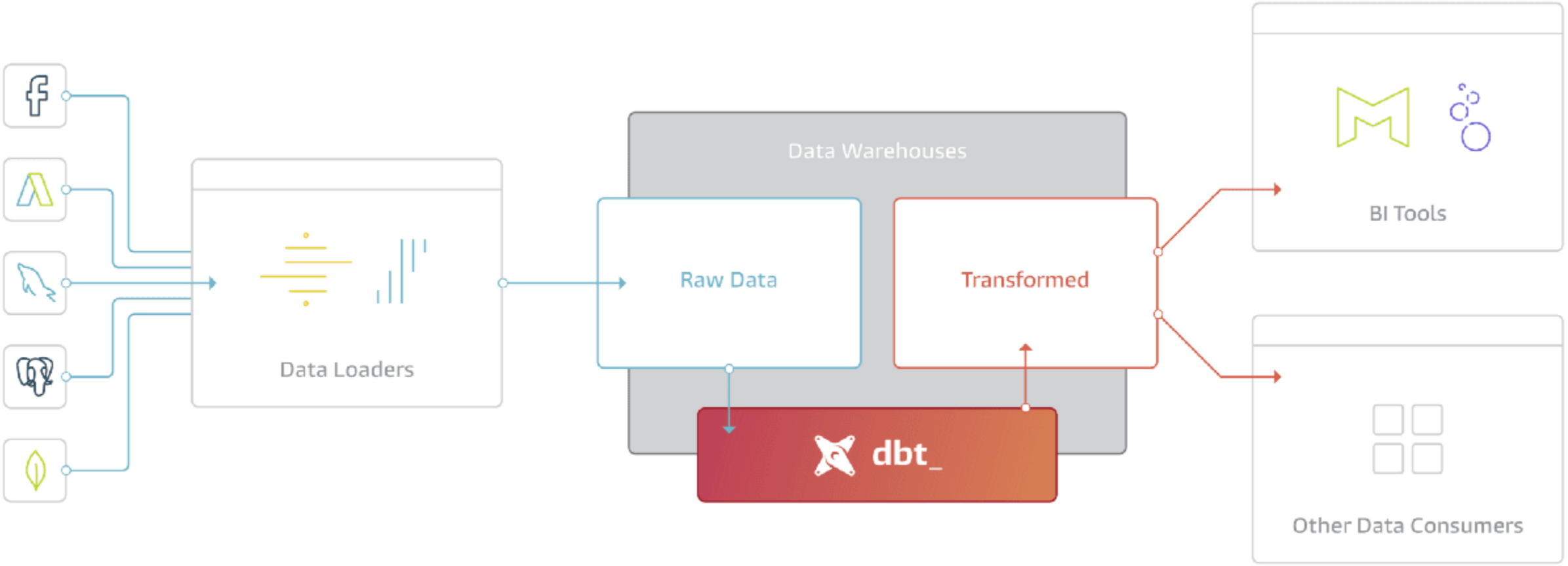
Type 2 keeps every version as a new row, the go-to choice for full history.

Type 3 adds a “previous value” column, good for simple before/after needs.

levi nine

Fundamentals – Data Build Tool

dbt Core


Data Warehouses
Raw Data
Transformed
dbt_
Data Loaders
BI Tools
Other Data Consumers

What is dbt?

"dbt enables analysts to work more like software engineers, using version control, testing, and modular SQL to build reliable data pipelines.“

T in ELT

dbt handles the Transform layer. Your data warehouse already has the data, dbt shapes it.

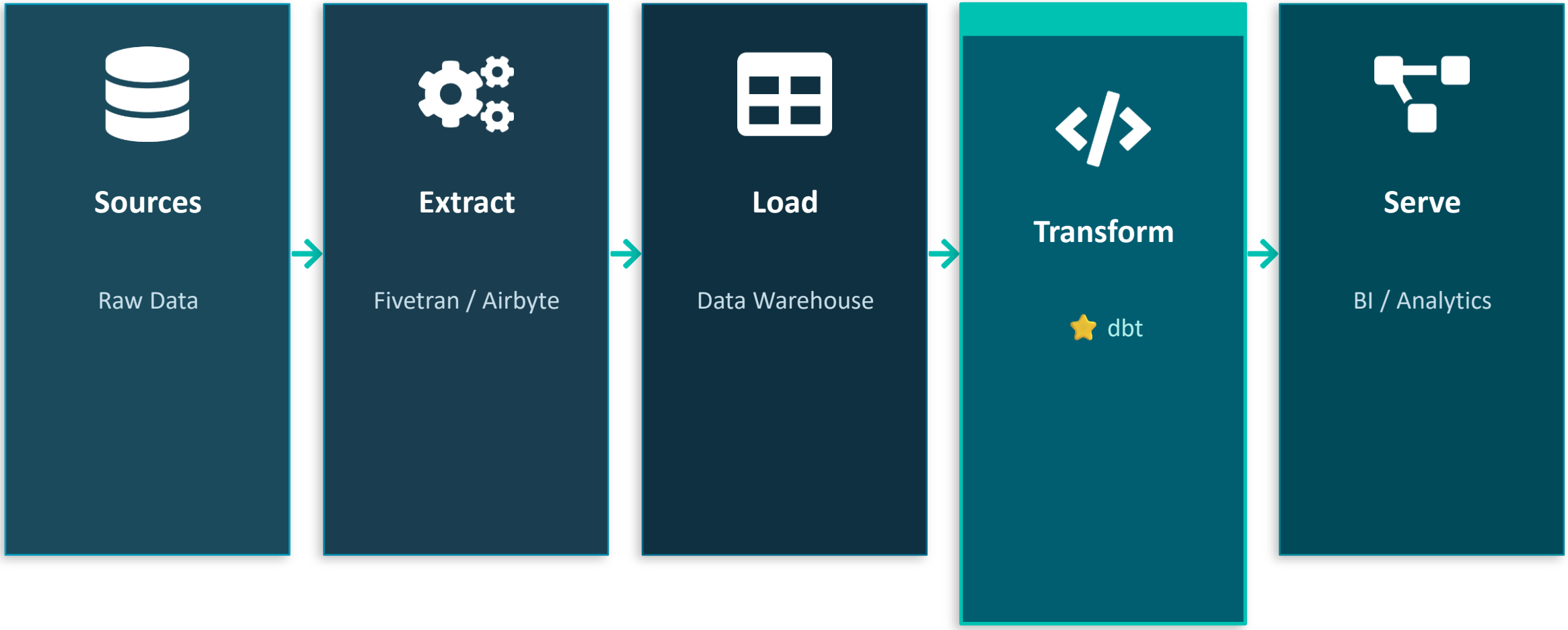
SQL-First

Write pure SQL. dbt compiles it, materializes it as tables or views, and tracks dependencies.

Software Practices

Versioning, testing, documentation and CI/CD, all applied to your data transformations.

dbt in Modern Data Stack



dbt lives in the Transform layer, working on data that's already loaded into your warehouse (BigQuery, Snowflake, Redshift, DuckDB…)

dbt Project Structure

```
my_dbt_project/
├── dbt_project.yml
├── profiles.yml
├── models/
│   ├── staging/
│   │   ├── stg_orders.sql
│   │   └── sources.yml
│   ├── intermediate/
│   └── marts/
│       └── fct_orders.sql
├── tests/
│   └── assert_*.sql
├── macros/
└── seeds/
```

dbt_project.yml

Project config: name, version, model paths, variable defaults, and materializations per folder.

models/staging/

Staging models clean raw source data. One model per source table, minimal transformations.

models/marts/

Business-level models. Fact and dimension tables consumed by BI tools.

sources.yml

Declares source tables and databases. Enables source freshness checks and testing.

macros/

Reusable Jinja SQL snippets and custom tests. DRY principle for SQL.

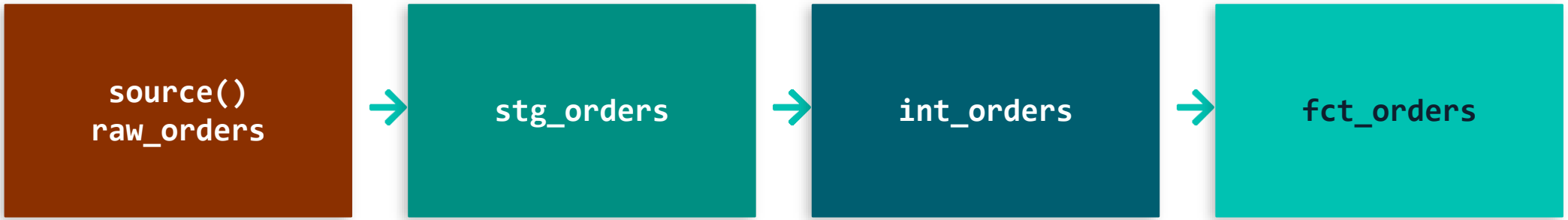
Jinja templates and Macros

Jinja templating

- allows you to embed programming logic within your SQL code, enabling you to create dynamic queries that can adapt based on conditions or inputs
- It enhances the capabilities of SQL by allowing for loops, conditionals, and variable definitions, making your dbt projects more flexible and powerful

Macros

- are reusable blocks of SQL code written using Jinja
- encapsulate complex logic that can be reused across multiple models, tests, or other macros
- the DRY (Don't Repeat Yourself) principle



Core Concept: Models

A Model is a .sql file

Each model = one SELECT statement.
dbt compiles it into a table or view.

models/stg_orders.sql

```sql
SELECT
    order_id,
    customer_id,
    status,
    created_at
FROM
    {{ ref('raw_orders') }}
```

Materialization Types

`view`

Virtual table, default. Runs query every time.

`table`

Physical table. Rebuilt from scratch each run.

`incremental`

Appends/merges only new rows. Efficient for large data.

`ephemeral`

CTE-like. Not persisted, used as building block.

DBT layers - models

Staging

the staging layer is the first place dbt touches your raw data.

Intermediate

this layer is where transformations become meaningful but are still reusable.

Marts

marts are the datasets analysts and dashboards actually use.

Key Functions: ref() and source()

{{ ref('model_name') }}

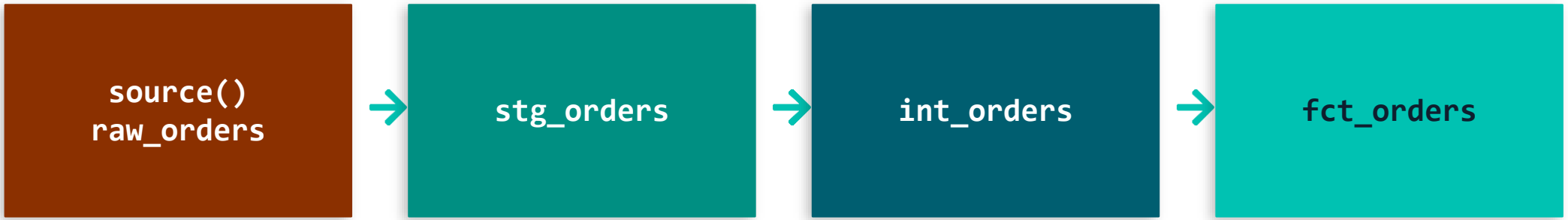
References another dbt model.

- Builds the dependency graph (DAG) automatically
- Resolves the right schema per environment
- Ensures models run in the correct order

{{ source('schema', 'table') }}

References raw source tables.

- Points to tables loaded by EL tools
- Defined in sources.yml config files
- Enables source freshness testing



Lineage

In dbt, lineage works by automatically building a **Directed Acyclic Graph (DAG)**.

- **ref() function:** When you write a model (e.g., stg_customers.sql) and use {{ ref('base_customers') }}, dbt captures this dependency and draws a direct link from base_customers to stg_customers

- **source() function:** When you reference raw data using {{ source('raw_data', 'orders') }}, dbt establishes the very first "dot" (origin) in your data flow.

Auditing projects

- auditing involves verifying data quality, historical changes, and pipeline accuracy

- dbt audits your data using testing, snapshots, and specialized packages.

Data engineering project audits ensure your pipelines are reliable, cost-effective, and secure. An audit maps your data flows, verifies data quality, checks for compliance, and removes wasteful cloud spending. It is a critical checkup to catch hidden errors before they damage your business

Snapshots

Snapshots:

In dbt Core, snapshots track data changes over time to create historical records.

The dbt snapshot command executes the Snapshots defined in your project. Snapshots record changes to your source data over time by implementing type-2 Slowly Changing Dimensions. Run dbt snapshot on a schedule (for example, daily) to capture changes in your source tables.

Packages:

Packages are reusable dbt project, like code libraries, that save you from building common tasks or macros from scratch

Testing Your Data

dbt has two types of tests to ensure data quality:

Schema Tests (schema.yml)

- **unique**
  Every value is distinct
- **not_null**
  No NULL values exist
- **accepted_values**
  Values are within a set
- **relationships**
  Foreign key integrity

Custom Tests (tests/*.sql)

```
-- tests/assert_order_total.sql

SELECT order_id

FROM {{ ref('fct_orders') }}

WHERE total_amount < 0

-- returns rows = FAIL
```

Any query returning rows = test failure.
Perfect for business-logic validations:

- Revenue cannot be negative
- Date ranges must be valid
- No duplicate transactions

.yml files but what are they?

The .yml files aren't defining structure or connections, dbt already knows your DAG from the ref()/source() calls in your SQL. What .yml files add is metadata: documentation, tests, and source definitions. So they're optional in the sense that dbt won't fail without them, but skipping them means you lose testing and docs coverage.

```yaml
sources:
  - name: nyc_taxi
    tables:
      - name: green_tripdata
        meta:
          external_location: "data/green_tripdata_2026-*.parquet"
      - name: yellow_tripdata
        meta:
          external_location: "data/yellow_tripdata_2026-*.parquet"
```

Essentials CLI Commands

`dbt run`

Execute all models. Materializes to your warehouse.

`dbt test`

Run all schema and custom tests.

`dbt build`

run + test + seed + snapshot in one command.

`dbt compile`

Compiles SQL without executing. Great for debugging.

`dbt docs generate`

Build the documentation site from your project.

`dbt docs serve`

Launch the local docs server in your browser.

`dbt run -s +my_model`

Run my_model and all its upstream dependencies.

`dbt source freshness`

Check how recently source tables were updated.

Essentials CLI Commands

Key Takeaways

- dbt transforms data already in your warehouse
- Models are .sql SELECT statements
- ref() and source() build the dependency DAG
- Project layers: staging → intermediate → marts
- Test data quality with schema & custom tests
- dbt run / test / build are your core commands

Next Steps

1. **Install dbt Core**
   pip install dbt-core dbt-<adapter>
2. **Run dbt init**
   Scaffold a new project interactively
3. **Write your first model**
   A simple SELECT from a source table
4. **Add a test**
   not_null + unique in schema.yml
5. **Explore dbt docs**
   docs.getdbt.com

More on this:

https://docs.getdbt.com/docs/introduction?version=2.0

levi nine

Docker and containerization

The problem that Docker solves

😫 "It Works on My Machine!"

- Different OS versions between dev and production
- Missing dependencies on the server
- "Works on my laptop, breaks in testing"
- Inconsistent environment configurations
- Hard to onboard new team members

🐋 Docker's Solution

- Package app + all dependencies together
- Run anywhere: laptop, server, cloud
- Same environment every time
- Easy to share and deploy
- Lightweight and fast to start

About Containers

💡 *Think of a container like a shipping container on a cargo ship, it holds everything needed to deliver your application, and it works the same way no matter where it's shipped.*

Isolated

Runs independently from other containers and the host OS. Your app's environment is fully contained.

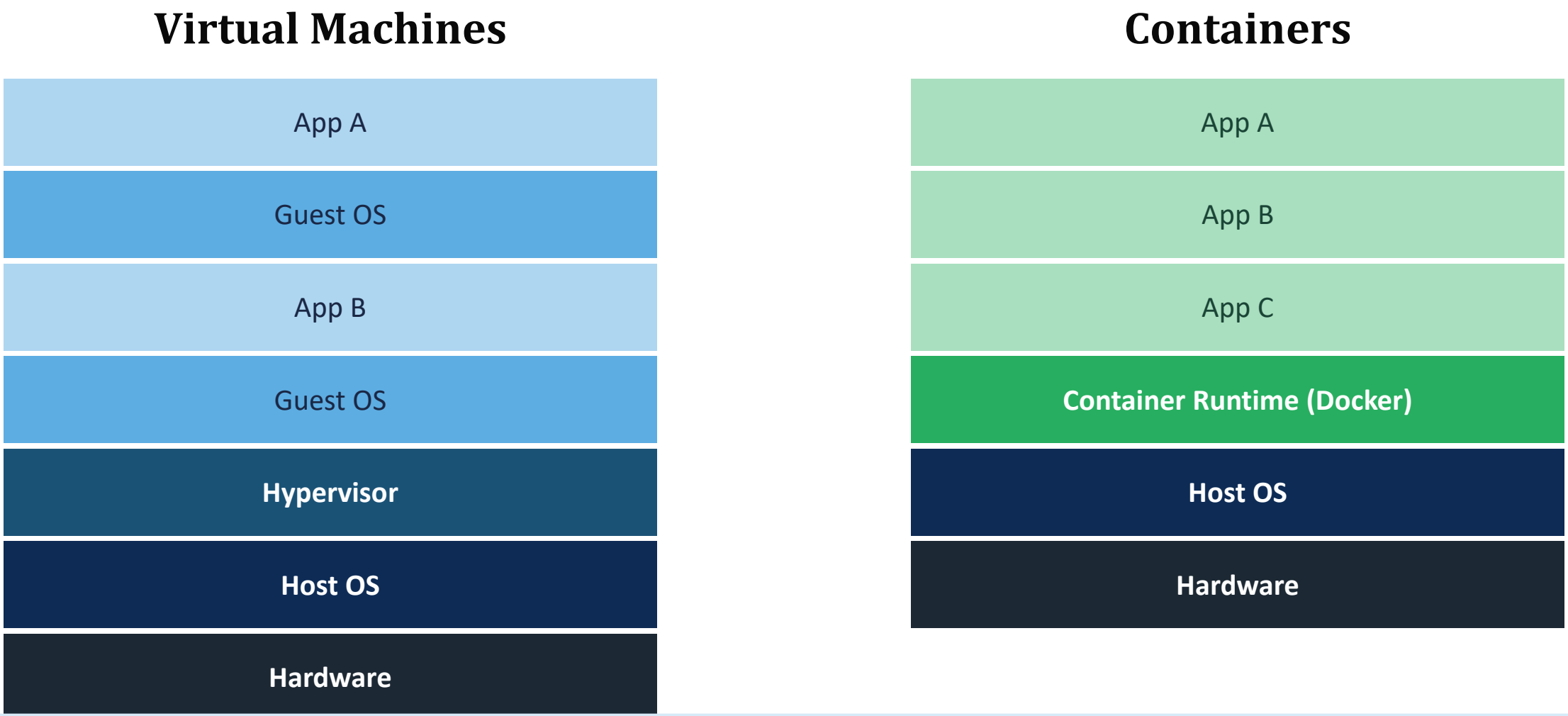
Portable

Build once, run anywhere. Works on Windows, Mac, Linux, and any cloud platform.

Lightweight

Shares the host OS kernel — much faster and smaller than full virtual machines.

Containers vs VMs



🚀 *Containers start in milliseconds and use MBs of RAM; VMs take minutes and use GBs*

Core Concepts

🖼️ Image

A read-only blueprint/template. Like a class in programming, defines what the container will look like.

📦 Container

A running instance of an Image. Like an object from a class, you can run many from one image.

📄 Dockerfile

A text file with step-by-step instructions to BUILD an image. Like a recipe.

🏪 Registry

A storage hub for images (e.g., Docker Hub). Push to share, pull to download images.

💾 Volume

Persistent storage attached to containers. Data survives even if the container stops.

🌐 Network

Allows containers to talk to each other and the outside world safely and efficiently.

Your first Dockerfile

Dockerfile

```
# 1. Start from a base image
FROM python:3.11-slim

# 2. Set working directory
WORKDIR /app

# 3. Copy files into the image
COPY requirements.txt .
COPY . .

# 4. Install dependencies
RUN pip install -r requirements.txt

# 5. Start the app
CMD ["python", "app.py"]
```

FROM: Base image from Docker Hub (so you don't start from scratch)

WORKDIR: Sets the folder inside the container for your app

COPY: Copies files from your computer into the image

RUN: Executes commands during build time

CMD: Command to run when container starts

Useful commands

Build `docker build -t myapp .`

Build an image from the Dockerfile in current directory

Run `docker run myapp`

Create and start a container from an image

Run `docker run -p 8080:80 myapp`

Run and map port 8080 (host) → port 80 (container)

Manage `docker ps`

List all currently running containers

Manage `docker ps -a`

List ALL containers (including stopped ones)

Manage `docker stop <id>`

Gracefully stop a running container

Images `docker pull nginx`

Download an image from Docker Hub registry

Images `docker images`

List all images stored locally on your machine

Automated Testing, Deployment, and Quality Gates

CI/CD for Data Pipelines

What is CI/CD?

CI/CD = Continuous Integration + Continuous Delivery/Deployment

Data Pipeline–Specific Considerations

- Test with representative data samples, not just code logic
- Data quality checks become part of the pipeline (Great Expectations, dbt tests)
- Schema migrations must be versioned and backward-compatible
- Backfill strategies needed when deploying pipeline changes to historical data

CI/CD Toolchain

📦 Version Control

Git + GitHub / GitLab / Bitbucket

⚙️ CI Platform

GitHub Actions · Jenkins · GitLab CI

🧪 Data Testing

Great Expectations · dbt tests · pytest

🏗️ Infrastructure as Code

Terraform · Pulumi · CloudFormation

🔄 Orchestration

Apache Airflow · Prefect · Dagster

📋 Artifact Registry

Docker Hub · ECR · Databricks Jobs

Config file Yaml for CI/CD Pipeline

```yaml
# .github/workflows/pipeline.yml
on: push:
  branches: [main, dev]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run data quality tests
        run: pytest tests/ --data-profile
      - name: Submit Spark job
        run: spark-submit bronze_ingestion.py
      - name: Validate Bronze output
        run: python validate_bronze.py
```

Real life scenarios

Microservices

Netflix, Spotify & Amazon run thousands of tiny services in containers. Each service scales independently.

CI/CD Pipelines

GitHub Actions and Jenkins use containers to run tests in clean, reproducible environments automatically.

Cloud Deployment

AWS, Google Cloud & Azure all run Docker containers. Deploy your app to any cloud with zero code changes.

Dev Environments

New team members get a full dev environment in minutes with 'docker compose up'. No manual setup!

Key takeways

1. Docker packages your app + all dependencies into a portable container
2. Containers are faster and lighter than Virtual Machines
3. A Dockerfile defines how to build your image step-by-step
4. Docker Hub is the public registry where you find and share images
5. Containers run consistently everywhere: dev, test, and production

levi nine

Q&A